COMPUTER LANGUAGES

Introduction

- Human languages are known as natural languages.
  - Unfortunately, computers can not understand natural languages (English, Gujarati, Spanish,.. etc),
  - as a result we must communicate with computers using computer languages (programming languages)

- Programming languages can be used to create programs that control the behavior of a computer and serve any purpose.
  - A programming language is a set of rules that provides a way of telling a computer what operations to perform.

Introduction

- English is a natural language. It has words, symbols and grammatical rules.
- A programming language also has words, symbols and rules of grammar.
- The grammatical rules are called syntax.
- Each programming language has a different set of syntax rules.

Levels of Programming Languages

High-level program

```
class Triangle {
  ...
  float surface()
    return b*h/2;
  }
```

Low-level program

```
LOAD r1,b
LOAD r2,h
MUL r1,r2
DIV r1,#2
RET
```

Executable Machine code

```
0001001001000101
0010010011101100
10101101001...
```

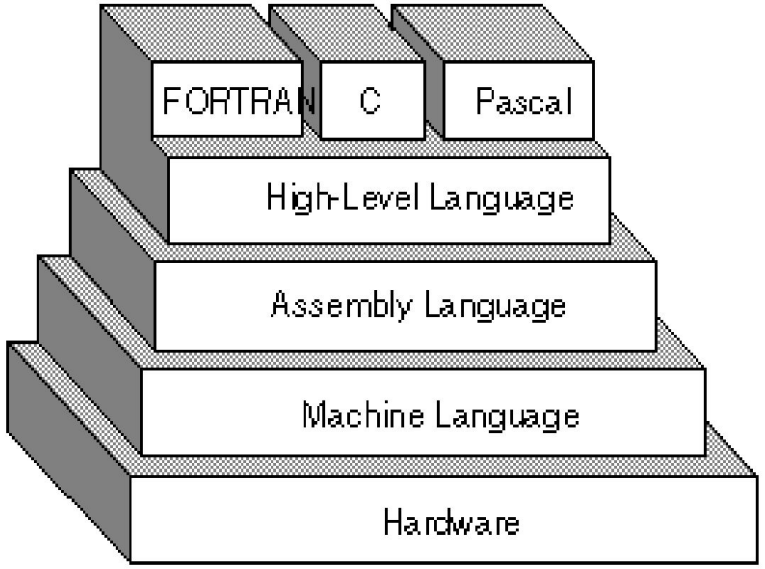
Architecture

Computer understand only binary . language (0 or 1)

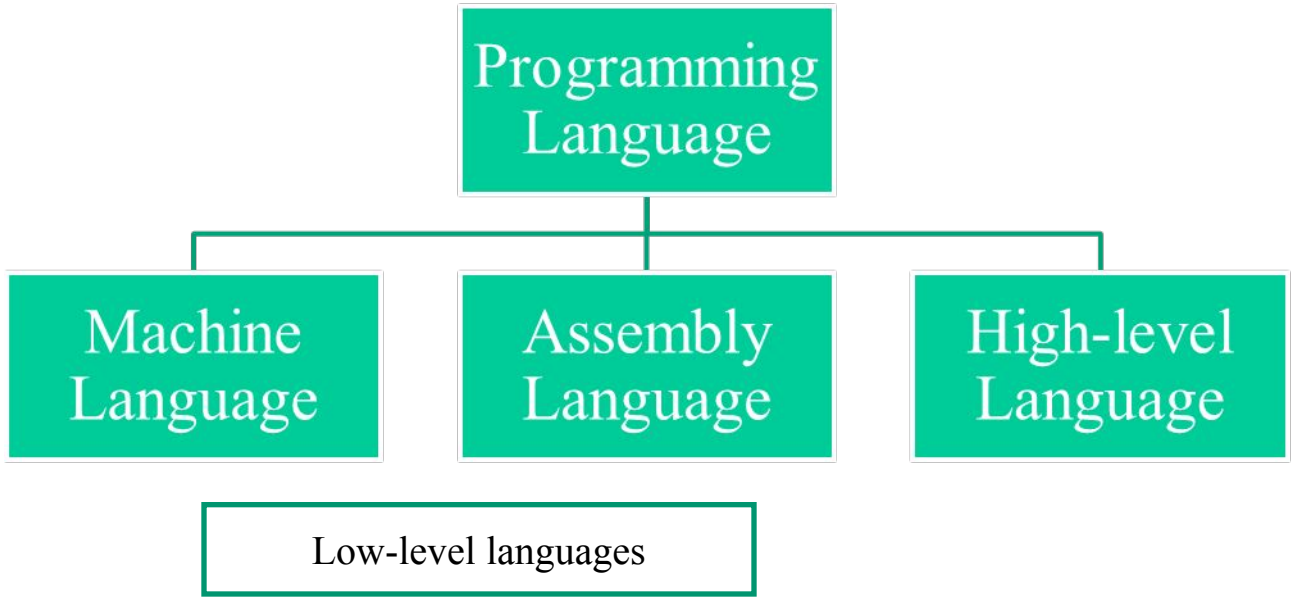
Binary language also known as machine or low level language

All the instructions given in binary form only – hard to understand by people

High-level language were developed



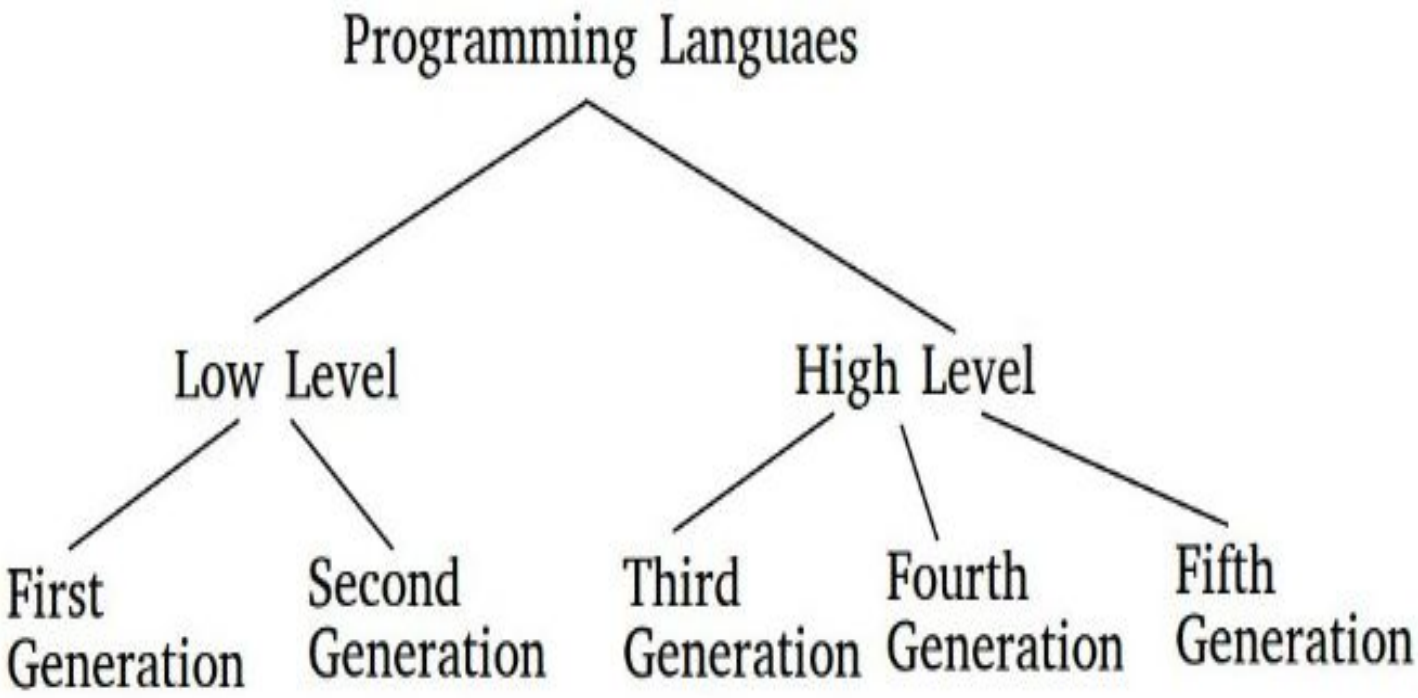
Classification of Programming Languages



Generations of Programming Languages

- Computer languages has the same history as the computers itself history.

- There are five generations of languages when programming method and techniques could be developed as far as development in hardware occurred.


Programming Languaes
Low Level
High Level
First Generation
Second Generation
Third Generation
Fourth Generation
Fifth Generation

First Generation (1GL) – Machine Language

- First-generation language was machine language
  - the level of instructions and data that the processor is actually given to work on **binary numbers 0s and 1s.**

- In the 1940s and 1950s, computers were programmed by scientists sitting before control panels equipped with toggle switches so that they could input instructions as strings of zeros and ones.

Machine Language – (1GL)

- Machine language Format
  - Operation code – instruct computer what functions are to be performed (such as addition or subtraction).
  - Operands – instruct the computer where to find or store the data on which the desired operation is to be performed
- Machine language is machine dependent as it is the only language the computer can understand.
  - Very efficient code but very difficult to write.



Machine Language – (1GL)

- **Advantages:**

1. Translation Free:
   - computer can directly execute without the need for conversion

2. High Speed:
   - Since no conversion is needed, applications developed using machine languages are extremely fast
   - Ex. Space control system, nuclear reactors and chemical processing

Machine Language – (1GL)

- **Disadvantages:**

1. **Machine Dependent**
   - Based on computer architecture,
   - application developed for one type of computer may not run on others
2. **Complex Language**
   - Difficult to read and write
3. **Error Prone**
   - Since programmer has to remember all the opcode and the memory locations, it is bound to be error prone
4. **Tedious**
   - Programming becomes too complex to modify

Second Generation (2GL) – Assembly Language

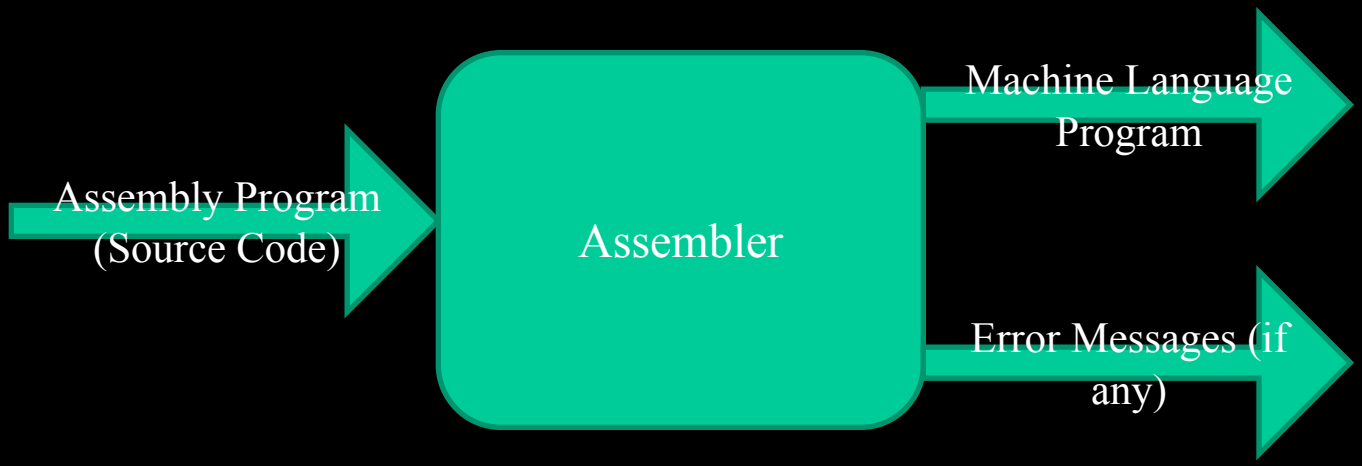
- By the late 1950s, this language had become popular.
- Known as Symbolic language.
- Assembly language consists of letters of the alphabet.
  - This makes programming much easier than trying to program a series of zeros and ones.
- An assembler converts the assembler language statements into machine language

Second Generation (2GL) – Assembly Language

- The general format of Assembly Language:

Label	Opcode	Operands	Comment
BEGIN	ADD	A, B	;Add B; to A

Second Generation (2GL) – Assembly Language



Second Generation (2GL) – Assembly Language

- **Advantages:**
- Easy to understand and Use
- Less Error Prone
- Efficiency
  - Faster compare to high-level language programs
- More control on hardware

Second Generation (2GL) – Assembly Language

- **Disadvantages:**

- Machine Dependent
  - Different computer have their own assembly languages
- Harder to learn
- Less efficient
- No standardization
- No support for modern software engineering technology

Third Generation (3GL)-High-level Language

- The third generation is also called procedural language /3 GL. It consists of the use of a series of English-like words that humans can understand easily, to write instructions. It's also called High-Level Programming Language
- Programmer can write program by learning syntax of language.

- High level language must use interpreter, compiler or translator to convert human understandable program to computer readable code (machine code).

```
X := X + Y ;
```

Third Generation (3GL)-High-level Language

- Many high level languages have appeared since Fortran II the most widely used have been:

COBOL Business applications
FORTRAN Engg & Scientific Applications
PASCAL General use and as a teaching tool
C & C++ General Purpose – currently most popular.
JAVA General all purpose programming

Advantages of High-level Languages

- Readability
  - Easy to read, write and maintain
- Machine independent
- Easy debugging
- Easier to maintain
- Low development cost
- Easy documentation

Disadvantages of High-level Languages

- Poor control on hardware
- Less efficient
  - Process of translation increases the execution time of an application

Fourth Generation (4GL)

- 4GLs have simple, English-like rules, commonly used to access databases
- 4GLs are divided into three categories:

1. Query Languages: allow user to retrieve information from databases (ex. SQL)
2. Report Generators: produce customized reports using data stored into databse
3. Application Generators: the user writes programs to allow data to be entered into the database

Fourth Generation (4GL)

- Advantage:

1. Easy to understand & learn.
2. Less time is required for application creation.
3. It is less prone to errors

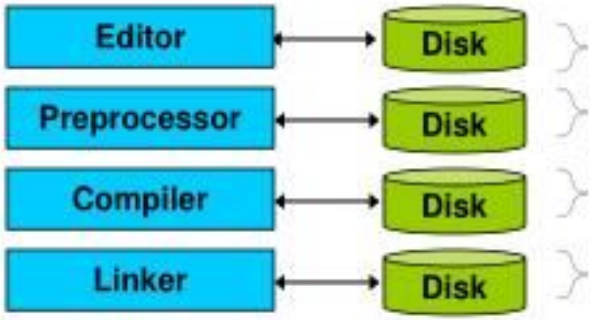
- Disadvantage:
  - Program need more disk space and large memory capacity compared to 3GL program

Fifth Generation Languages (5GL)

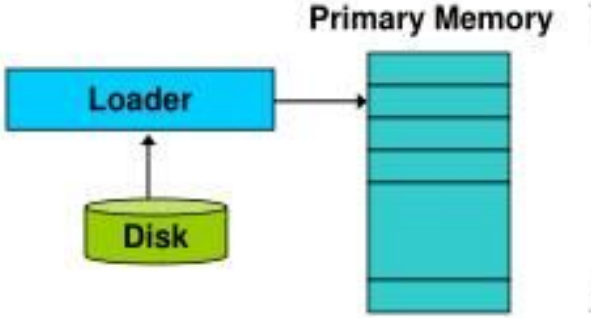
- It is based on the concept of artificial intelligence. It uses the concept that rather than solving a problem algorithmically.
- Though no clear definition at present, natural language programs generally can be interpreted and executed by the computer with no other action by the user than stating their question.
- User will free from learning any programming language to communicate with computes
- Programmers may simply type the instruction or tell the computer via microphones what is needs to do
- Limited capabilities at present.

A Typical C Program Development Environment

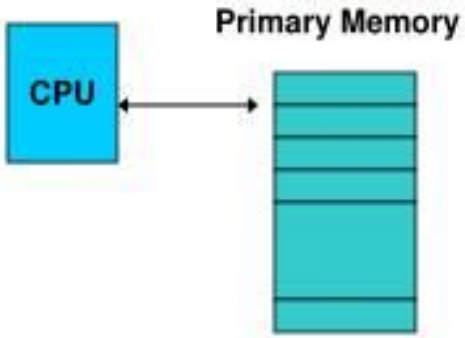
- **Phases of C Programs:**



1. Program is created in the editor and stored on disk
2. Preprocessor program processes the code
3. Compiler creates object code and stores it on disk.
4. Linker links the object code with the libraries



5. Loader puts program in memory.



6. CPU takes each instruction and executes it, possibly storing new data values as the program executes

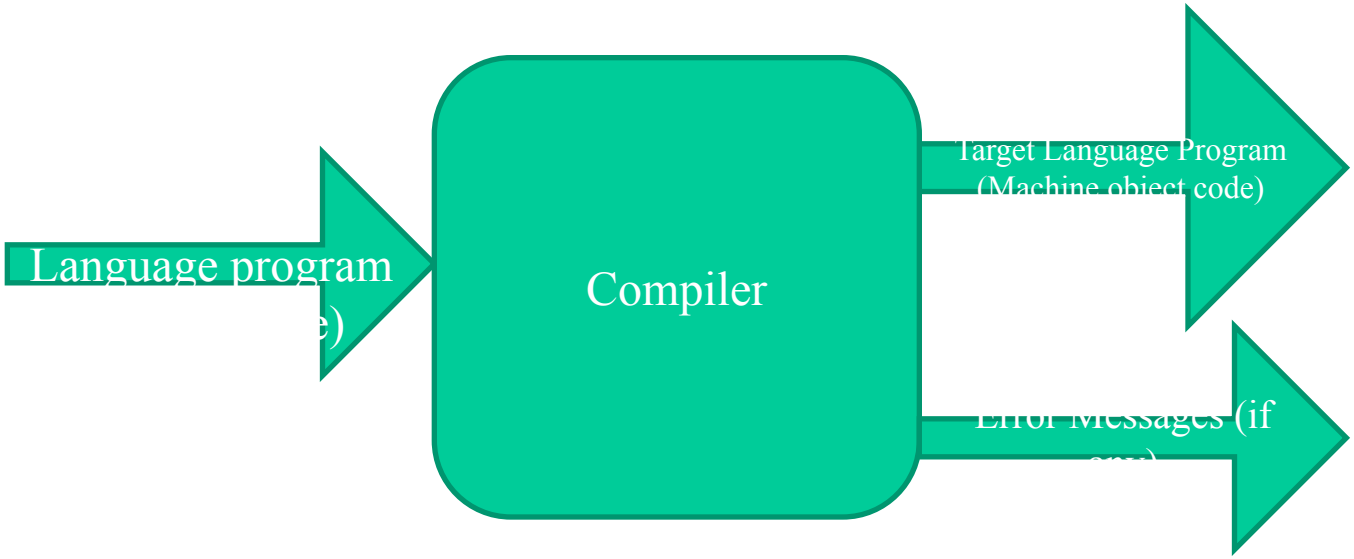
1. *Edit*
2. *Preprocess*
3. *Compile*
4. *Link*

---

5. *Load*
6. *Execute*

Compiler

- Language translator: convert high-level language into machine language
- Compiler replaces single high-level statement with a series of machine language instructions



Compiler

- Program compilation: Compiler translates whole program into an equivalent machine language program
- Once the program has been compiled, the resulting machine code (object code) saved separately, which can be run on its own at any time
- Once the object code is generated, there is no need of actual source code
- If source code is modified – necessary to recompile the program

- For each high-level language, a separate compiler is required

Interpreter

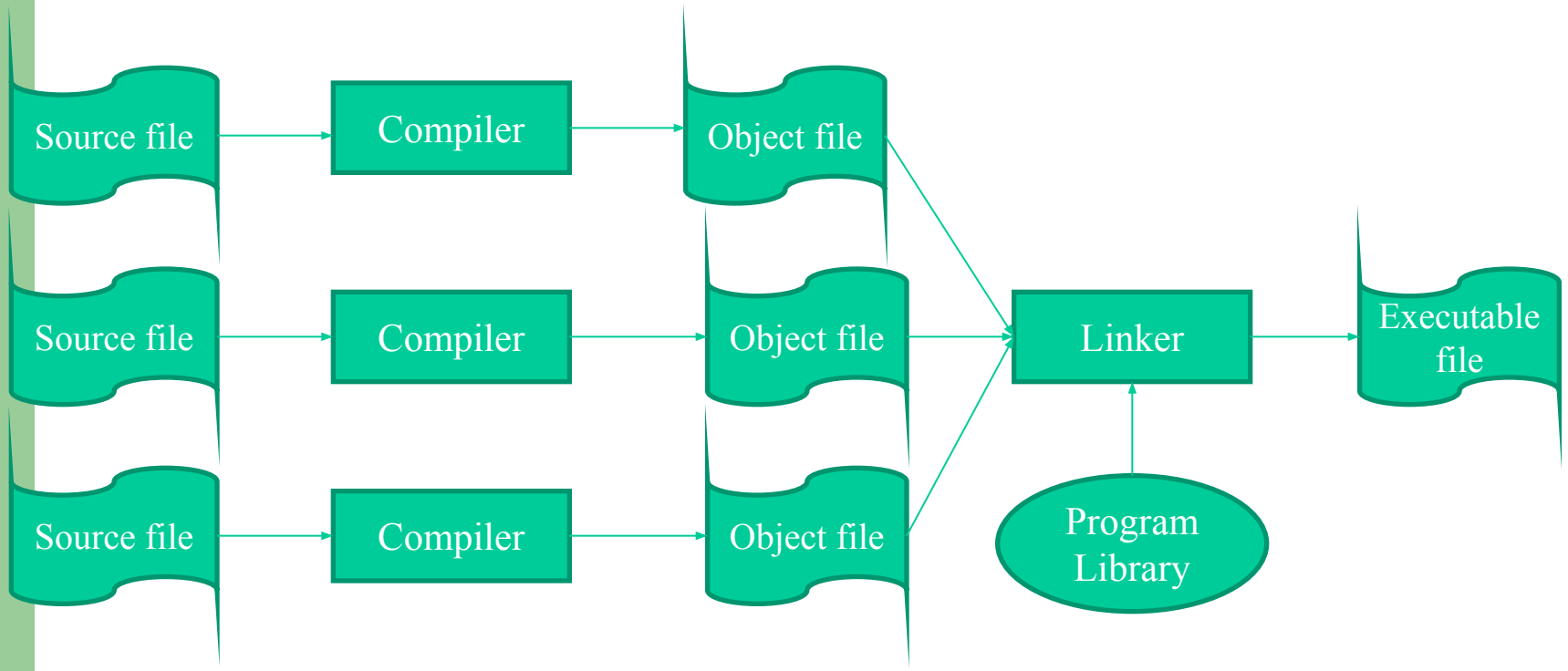
- Language translator: convert high-level language into machine language
- Unlike compiler: it translates a statement of program and executes immediately, before translating next statement



Linker

- An application usually consists hundreds, thousands or even million of lines of code
- Code divided into logical groups and stored into different modules
  - So debugging and maintenance becomes easy
  - Each module can be modified and compiled independently
  - Linker links several object modules and libraries to form a single program

Linker


Source file
Compiler
Object file
Source file
Compiler
Object file
Linker
Executable file
Source file
Compiler
Object file
Program Library

Loader

- The loader is the program of the operating system which **loads the executable from the disk into the primary memory(RAM) for execution**. It allocates the memory space to the executable module in the main memory and then transfers control to the beginning instruction of the program.
- The loader is an important part of the system, as it loads the program, places them in the memory, and prepares them for the final execution. Loading a program includes reading the content of the executable file that has program instructions from the memory, then it performs some other preparatory task to execute the task.

Loader

- Types of loader:
- Absolute Loader:
  - Loads the file into memory at the location specified by the beginning portion (header) of file and then passes control to program
  - If memory space specified by header is currently in use, execution cannot be processed
    - User must wait until requested memory becomes free
- Relocating loader:
  - A relocating loader can load the program into different regions of memory at different times during execution, allowing better utilization of the system's memory

Characteristics of good programming

- **What is a Good Program** ?
  - A Good Program means that it should produce correct and faster results, taking into account all the memory constraints.
  - The following is the list of good programming habits that one should keep in mind while writing:
    - Clarity and Simplicity of Expression
    - Use of proper names for identifiers
    - Comments
    - Indentation

- **Clarity and Simplicity**
  - Expressions are use to implement a particular task. It is a combination of Operators, Operands, and Constants. Any expression used in the program should be understood by the user.
  - For example-
  - cube= x*x*x we can use library function and can make it shorter. Cube=pow(x,3)

- Clarity

X = A+B/A-B – U +VY/X+Y

We may simplify the above expression and write:

X1 = (A+B) / (A-B)

X2 = (U+V*Y) / (X +Y)

X = X1 –X2

..Cont

- **Use of proper names for identifiers** :
  - Identifiers are user defined names.
  - They are use to name things. A name is associated with a function or data object (constants and variables) and used to refer to that function or data object.

…Cont

- **Comments** :
  - They are usually added with the purpose of making the source code easy to understand. Hence, add comments to your code in simple English language that describes the function of the code and the reason
- **Indentation :**
  - Leading white space (spaces and tabs) at the beginning of each statement, which is used to determine the group of statements, is known as "indentation".

…Cont

- The quality of the program depends upon the instructions given to it. However, it is required to provide the proper and correct instructions to the computer in order to provide a correct and desired output A computer program should have some important characteristics, which are as follows :
- 1) Flexibility 2) Portability 3) User Friendly 4) Reliability 5) Self-Documenting Code

…Cont

1. **Flexibility**:

   A program should be flexible enough to handle most of the changes without having to rewrite the entire program.

2. **User Friendly**:

   A program that can be easily understood by a beginner is called **user friendly .**In addition, the proper message for the user to input data and to display the result, besides making the program easily understandable and modifiable

…Cont

3.Portability:

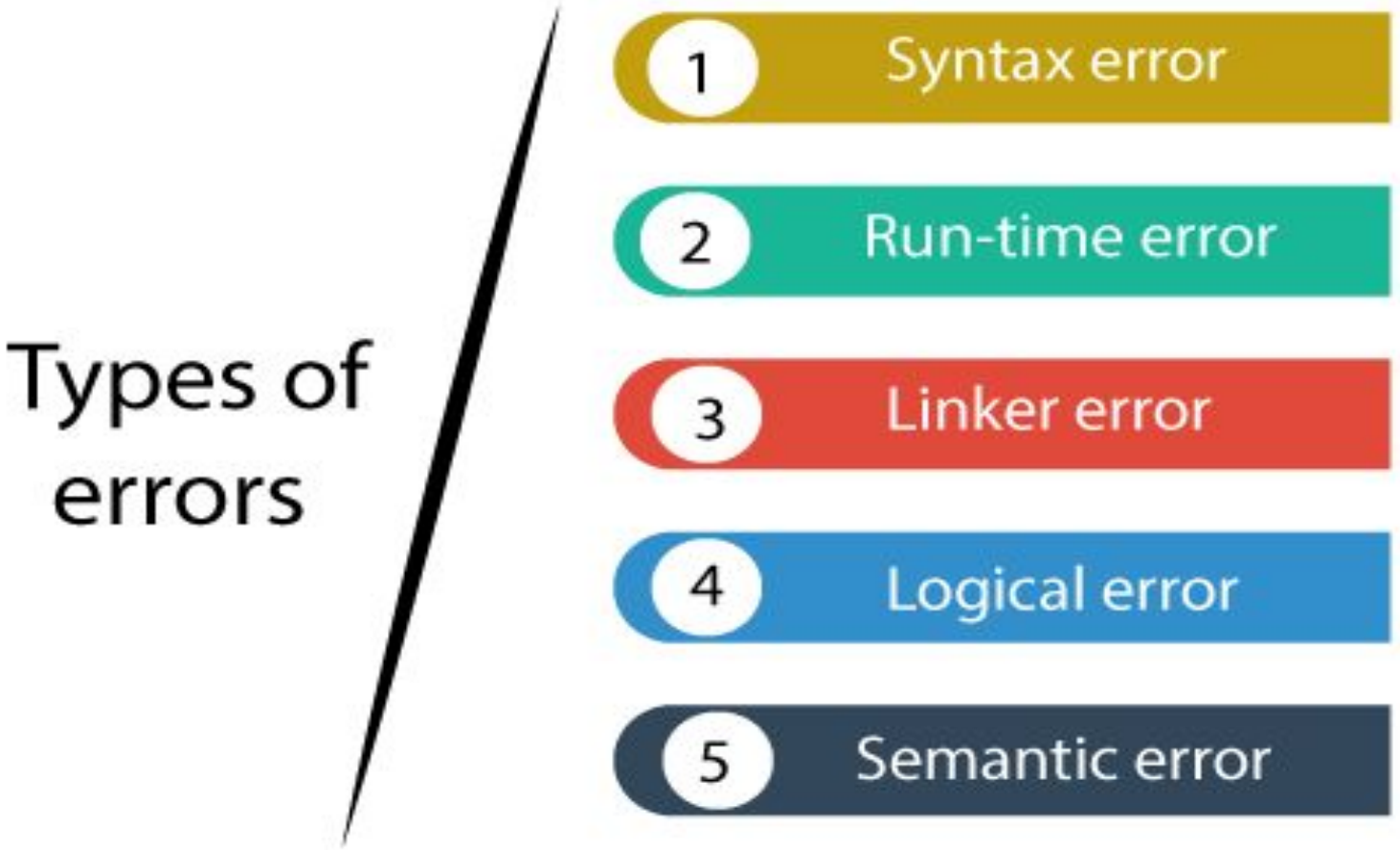
- Portability refers to the ability of an application to run on different platforms (operating systems) with or without minimal changes. Since the change of platform is a common phenomenon nowadays, due to the developments in hardware and the software, portability has to be taken care of it.
- High language programs are often more portable than assembly language programs

- **4) Reliability**:
  - It is the ability of a program to do its intended function accurately even if there are even small changes in the computer system.
  - Moreover, the program must be able to handle unexpected situations like wrong input or no input.
    For example, if the user does/gives wrong information to input, it should display a proper error message.

…Cont

- **5) Self-Documenting Code**:
  - The source code, which uses a suitable name for the identifiers (variables and methods), is called the self-documenting code.
  - Also, giving a proper name for variables and methods would tell the reader of your code clearly -what is it doing? Hence, a good program must have a self-documenting code

Program Errors



Syntax error

- Syntax errors are also known as the compilation errors as they occurred at the compilation time, or we can say that the syntax errors are thrown by the compilers. These errors are mainly occurred due to the mistakes while typing or do not follow the syntax of the specified programming language. These mistakes are generally made by beginners only because they are new to the language. These errors can be easily debugged or corrected.

If we want to declare the variable of type integer,

```
int a; // this is the correct form

Int a; // this is an incorrect form.
```

Commonly occurred syntax errors are:

- If we miss the parenthesis (}) while writing the code.
- Displaying the value of a variable without its declaration.
- If we miss the semicolon (;) at the end of the statement.

Run-time error

- Sometimes the errors exist during the execution-time even after the successful compilation known as run-time errors. When the program is running, and it is not able to perform the operation is the main cause of the run-time error. The division by zero is the common example of the run-time error. These errors are very difficult to find, as the compiler does not point to these errors

Linker error

- Linker errors are mainly generated when the executable file of the program is not created. This can be happened either due to the wrong function prototyping or usage of the wrong header file.

```
#include <stdio.h>
int Main()
{
   int a=78;
   printf("The value of a is : %d", a);
   return 0;
}
```

Logical error

- The logical error is an error that leads to an undesired output. These errors produce the incorrect output, but they are error-free, known as logical errors. These types of mistakes are mainly done by beginners.

```
#include <stdio.h>
int main()
{
   int sum=0; // variable initialization
   int k=1;
   for(int i=1;i<=10;i++);
   {
      sum=sum+k;
      k++;
   }
printf("The  value of sum is %d", sum);
    return 0;
}
```

Semantic error

- Semantic errors are the errors that occurred when the statements are not understandable by the compiler.
  - Use of a un-initialized variable.

```
int i;
i=i+2;
```

  - Type compatibility

```
int b = "javatpoint";
```

  - Errors in expressions

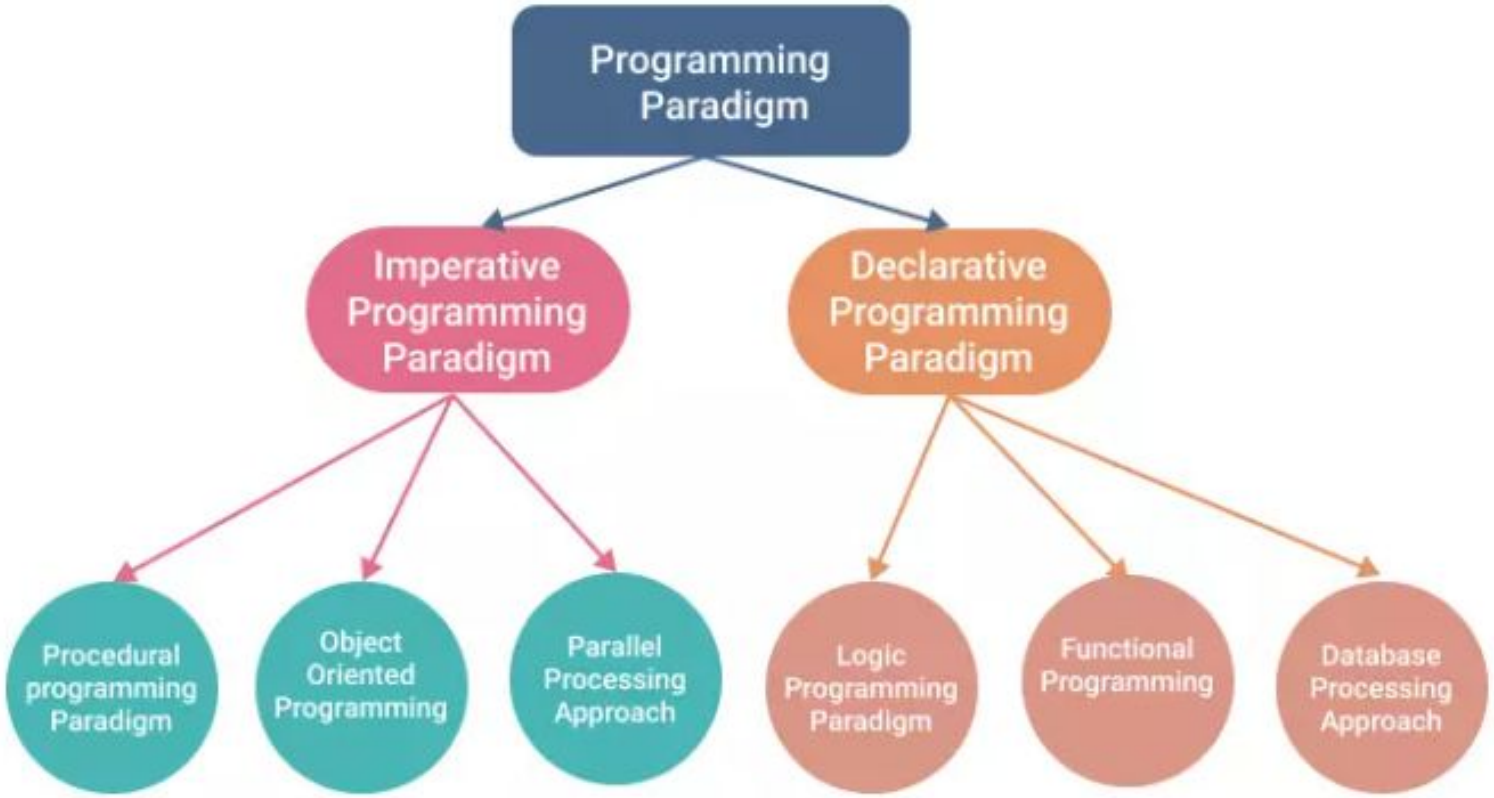
```
int a, b, c;
a+b = c;
```

  - Array index out of bound

```
int a[10];
a[10] = 34;
```

?What is programming paradigm

- A ***programming paradigm*** is a way, an approach, a style by which we write programs in a specific programming language to solve some problem.
- It is the approach creation of objects and creating programs in **object oriented programming** or
- Solving problems with series of functions in **functional programming** or
- A style of creation of logic by series of symbols in **logic programming**


Programming Paradigm
Imperative Programming Paradigm
Declarative Programming Paradigm
Procedural programming Paradigm
Object Oriented Programming
Parallel Processing Approach
Logic Programming Paradigm
Functional Programming
Database Processing Approach

Imperative Languages

- Oldest paradigm
- Programs are executed step by step
- Change the current state of the program.
- For examples C,C++,Java, Pascal

Procedural programming

- Basically, procedural programming revolves around a sequence of instructions given to the computer. These sequences of instructions are known as **procedures**.
- Since the instructions are carried out in well-defined sequences, we say that procedural programming takes a **top-down approach**.

Object Oriented Programming (OOP)

- This is used to work on real-world entities in form of class and objects. Class is the blueprint of the object and you can replicate as many as the object you want. These classes contain some properties and method which all are replicated in objects. Example **C++**, **Python**, etc

Example in Python

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

Parallel processing

- In this type of programming, a program is processed by dividing it into multiple processors. The system contains multiple processors to solve the problem in less time.

Declarative programming paradigm

- A ***declarative programming paradigm*** are those paradigms in which the programmer describes the property of the result **without focusing on how to achieve it** (imperative programming focuses on how to achieve the task by changing the state of the program).
- For Example Prolog,javascript,SQL

Logic programming paradigm

- Logic programming uses sentences in logical form and creates an expression by using symbols.

```
    int sumOfCube=0,a,temp;
    int number=153;
    temp=number;
    while(number>0)
    {
        a=number%10; //Getting last digit
        number=number/10;  //Removing last digit from number
        sumOfCube=sumOfCube+(a*a*a); //Calculating sum of cubes of digits
    }
    if(temp==sumOfCube)
        System.out.println(temp+" is an armstrong number");
    else
        System.out.println(temp+ "is not an armstrong number");
    }
}
```

Functional Languages

- Functional programming is the programming in which a program is constructed by creating and using functions.
- Rather than a series of statements functional programming use function to map and change one value to another value
- ML, Scheme and LISP are examples of functional languages.

```c
int myFunction(int x, int y) {
  return x + y;
}

int main() {
  int result = myFunction(5, 3);
  printf("Result is = %d", result);
  return 0;
}
// Outputs 8 (5 + 3)
```

Program Documentation

- The program documentation is a kind of documentation that gives a comprehensive procedural description of a program.
- It shows as to how software is written.
- The program documentation describes what exactly a program does by mentioning about the requirements of the input data and the effect of performing a programming task.
- Documentation are needed to ensure that we are able to communicate effectively, with all the people who will be involved in the development of the program or system.
- A good documentation will help the user to save their time from spending lots of unnecessary time trying to figure out how to use your program

Characteristics of Good Documentation

A good documentation is:

- Accessible
- Readable
- Understandable
- Correct
- Controlled

Testing and Debugging

- Testing:
  - Process of verifying and validating.
  - Check technical requirements as guided by its design and development.
  - Meets the user requirements effectively and efficiently by handling all the exceptional and boundary cases.

Testing and Debugging

- **Debugging:**
  - Process of fixing a bug in the software.
  - identifying, analyzing, and removing errors.
  - It is considered to be an extremely complex and tedious task because errors need to be resolved at all stages of debugging.

Operating Systems

Introduction to Operating System (OS)

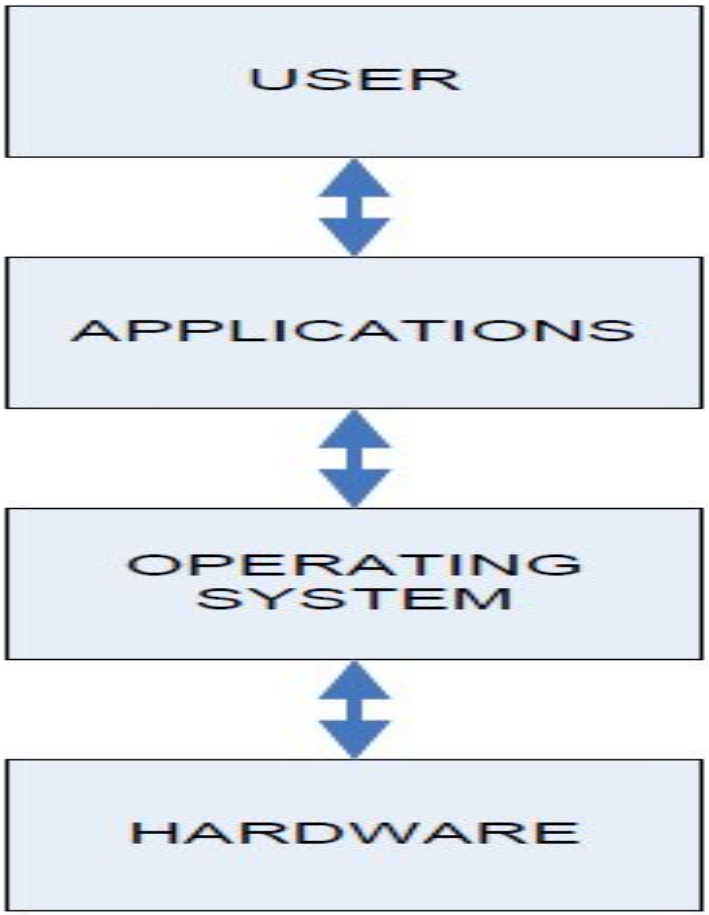
What is an Operating System (1)?

- A modern computer consists of:
  - One or more processors
  - Main memory
  - Disks
  - Printers
  - Various input/output devices.
- Managing all these varied components requires a layer of software – the **Operating System (OS).**

?What is an Operating System

- An **Operating System (OS)** is a software that acts as an interface between computer hardware components and the user.
- Every computer system must have at least one operating system to run other programs. Applications like Browsers, MS Office, Notepad Games, etc., need **some environment** to run and perform its tasks.
- It is not possible for the user to use any computer or mobile device without having an operating system.

The Structure of Computer Systems



.Accessing computer resources is divided into *layers*

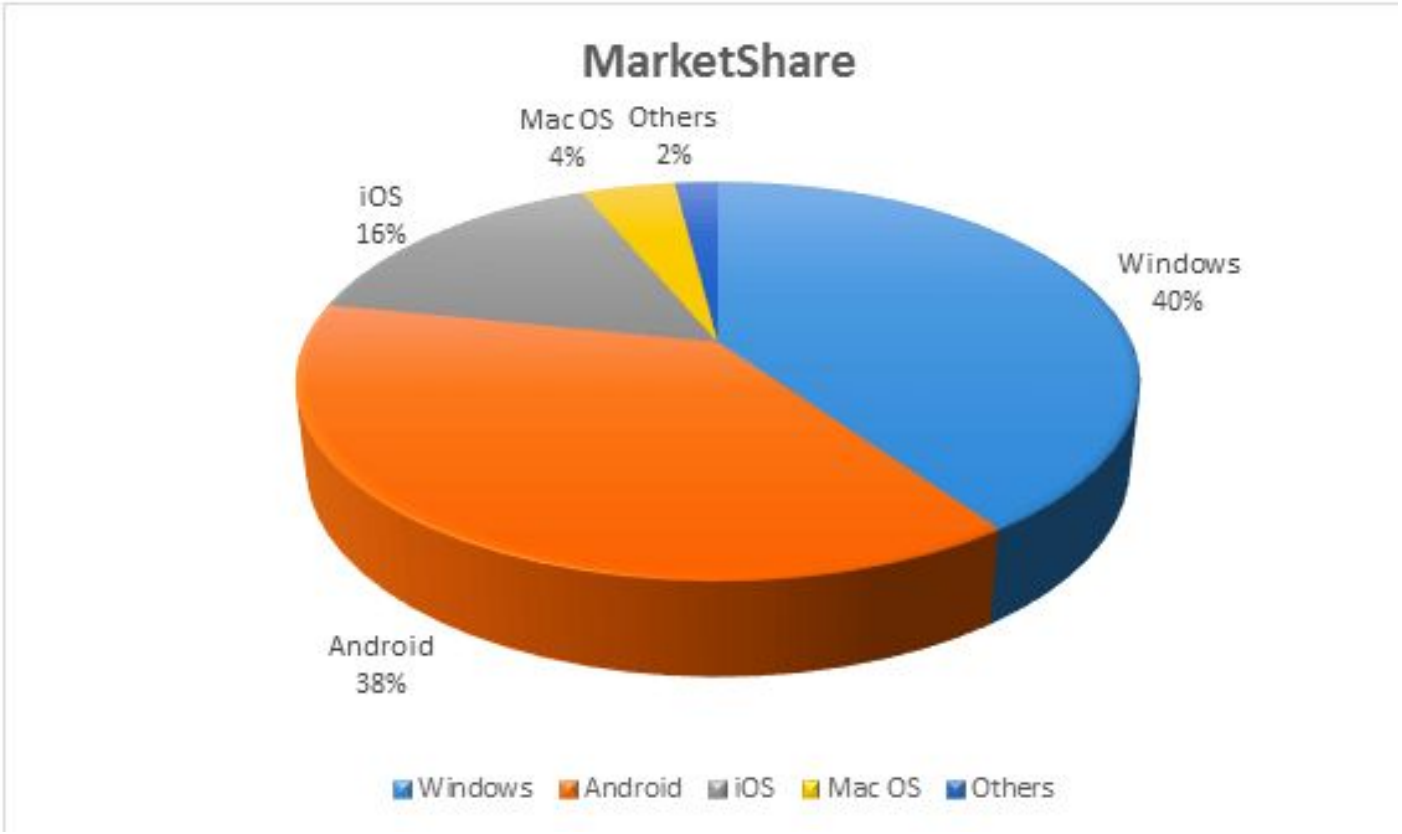
.Each layer is isolated and only interacts directly with the layer below or above it

If we install a new hardware device
.No need to change anything about the user/applications
.However, you do need to make changes to the operating system
You need to install the device drivers that the operating system will use to control the new
.device

If we install a new software application
.No need to make any changes to your hardware
But we need to make sure the application is supported by the operating system
.user will need to learn how to use the new application

If we change the operating system
Need to make sure that both applications and hardware will compatible with the new operating
.system

Operating System with Market Share



CPU – Central Processing Unit

.This is the brain of your computer

.It performs all of the calculations

In order to do its job, the CPU needs commands to perform, and data to .work with

.The instructions and data travel to and from the CPU on the system bus

The operating system provides rules for how that information gets back and . forth, and how it will be used by the CPU

RAM – Random Access Memory

- This is like a desk, or a workspace, where your computer temporarily stores all of the information (data) and instructions (software or program code) that it is currently using.
- Each RAM chip contains millions of address spaces.
- Each address space is the same size, and has its own unique identifying number (address).
- **The operating system provides the rules for using these memory spaces, and controls storage and retrieval of information from RAM.**
- Device drivers for RAM chips are included with the operating system.

Operating System Mode

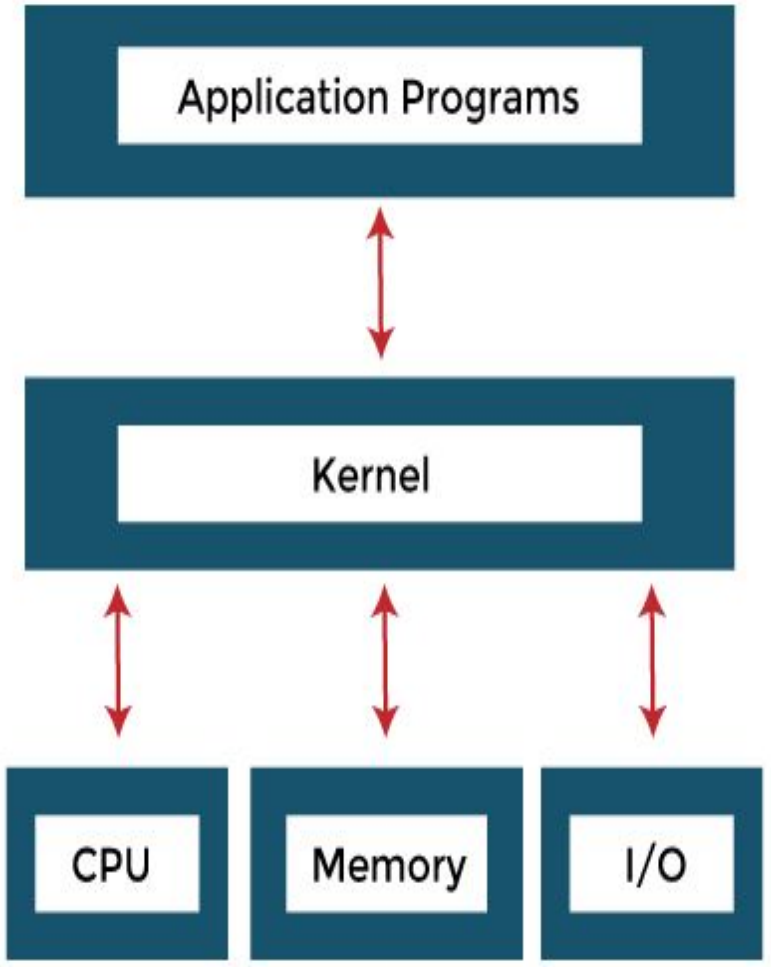
- We have two modes of the operating system: ***user mode*** and ***kernel mode***.
- **Mode bit** is required to identify in which particular mode the current instruction is executing. If the mode bit is 1, it operates user mode, and if the mode bit is 0, it operates in kernel mode.

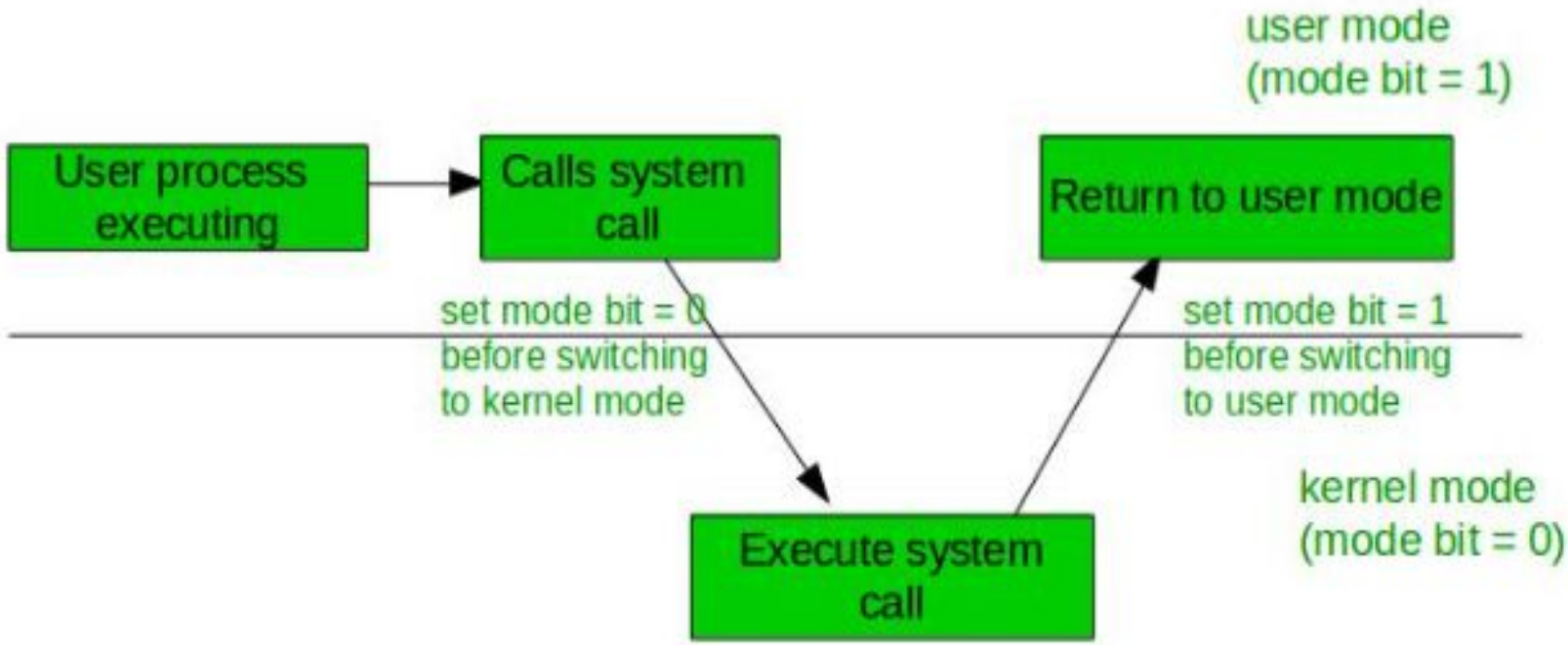
Operating System Mode

The *User Mode* is concerned with the actual interface between the user and the .system
It controls things like running applications .

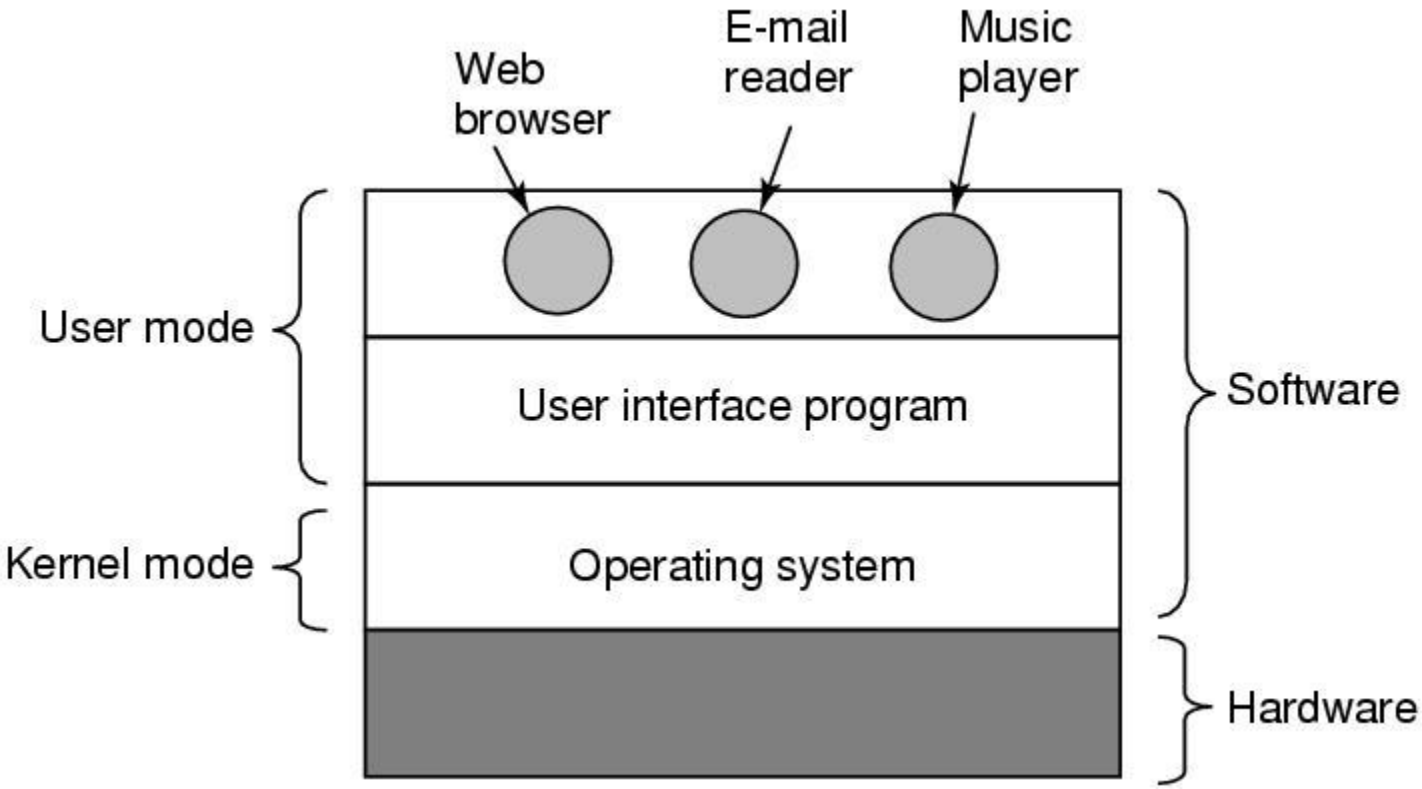
The *Kernel Mode* is concerned with .everything running in the background
It controls things like accessing system resources, controlling hardware functions
In kernel mode, the program has direct and unrestricted access to system resources.and accessing files and .processing program instructions

System calls are used to change .mode from User to Kernel




user mode
(mode bit = 1)
User process executing
Calls system call
Return to user mode
set mode bit = 0
before switching
to kernel mode
set mode bit = 1
before switching
to user mode
kernel mode
(mode bit = 0)
Execute system call

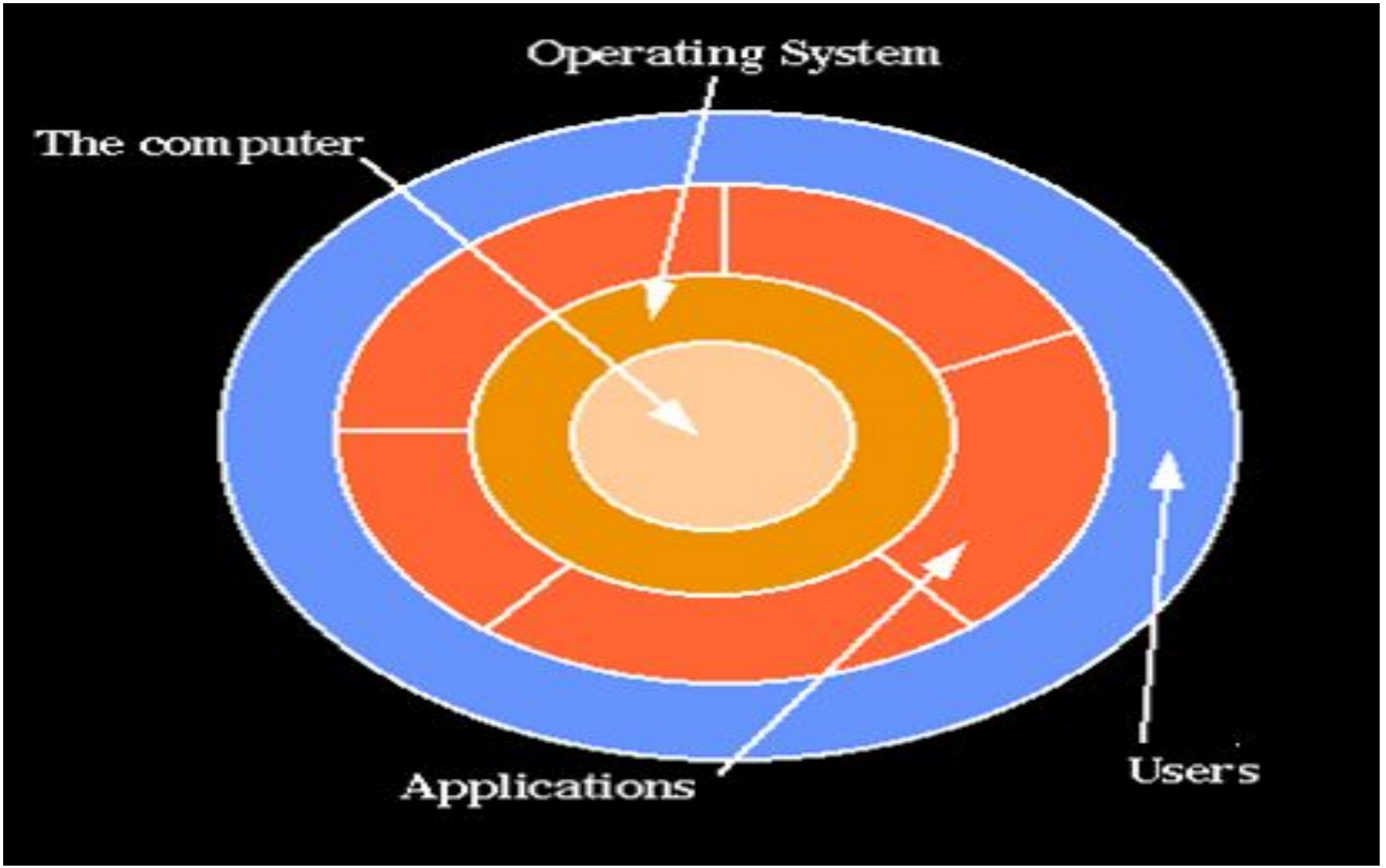
?Where does the OS fit in



?Why are Operating Systems Used

- It controls all the computer resources.
- It provides valuable services to user programs.
- It coordinates the execution of user programs.
- It provides resources for user programs.
- It provides an interface (virtual machine) to the user.
- It hides the complexity of software.
- It supports multiple execution modes.
- It monitors the execution of user programs to prevent errors.

Hierarchical view of computer system



Evolution of Operating Systems

- **No OS – (0s to 1940s) –**
  - As we know that before 1940s, there was no use of OS . Earlier, people are lacking OS in their computer system so they had to manually type instructions for each tasks in machine language(0-1 based language) . And at that time , it was very hard for users to implement even a simple task.

Batch Processing Systems -(1940s to :1950s)

- With the growth of time, batch processing system came into the market .Now Users had facility to write their programs on punch cards and load it to the computer operator. And then operator make different batches of similar types of jobs and then serve the different batch(group of jobs) one by one to the CPU .CPU first executes jobs of one batch and them jump to the jobs of other batch in a sequence manner

Multiprogramming Systems -(1950s to : 1960s)

- Multiprogramming was the first operating system where actual revolution began.It provide user facility to load the multiple program into the memory and provide a specific portion of memory to each program. When one program is waiting for any I/O operations (which take much time) at that time the OS give permission to CPU to switch from previous program to other program(which is first in ready queue) for continuos execution of program with interrupt.

Time-Sharing Systems -(1960s to 1970s)

- Time-sharing systems is extended version of multiprogramming system. Here one extra feature was added to avoid the use of CPU for long time by any single program and give access of CPU to every program after a certain interval of time. Basically OS swithces from one program to another program after a certain interval of time so that every program can get access of CPU and complete their work.

Introduction of GUI -(1970s to 1980s)

- With the growth of time, Graphical User Interfaces (GUIs) came. First time OS became more user-friendly and changed the way of people to interact with computer. GUI provides computer system visual elements which made user's interaction with computer more comfortable and user-friendly. User can just click on visual elements rather than typing commands. Here are some feature of GUI in Microsoft's windows icons, menus and windows.

Networked Systems – (1980s to 1990s)

- At 1980s,the craze of computer networks at it's peak .A special type of Operating Systems needed to manage the network communication . The OS like Novell NetWare and Windows NT were developed to manage network communication which provide users facility to work in collaborative environment and made file sharing and remote access very easy.

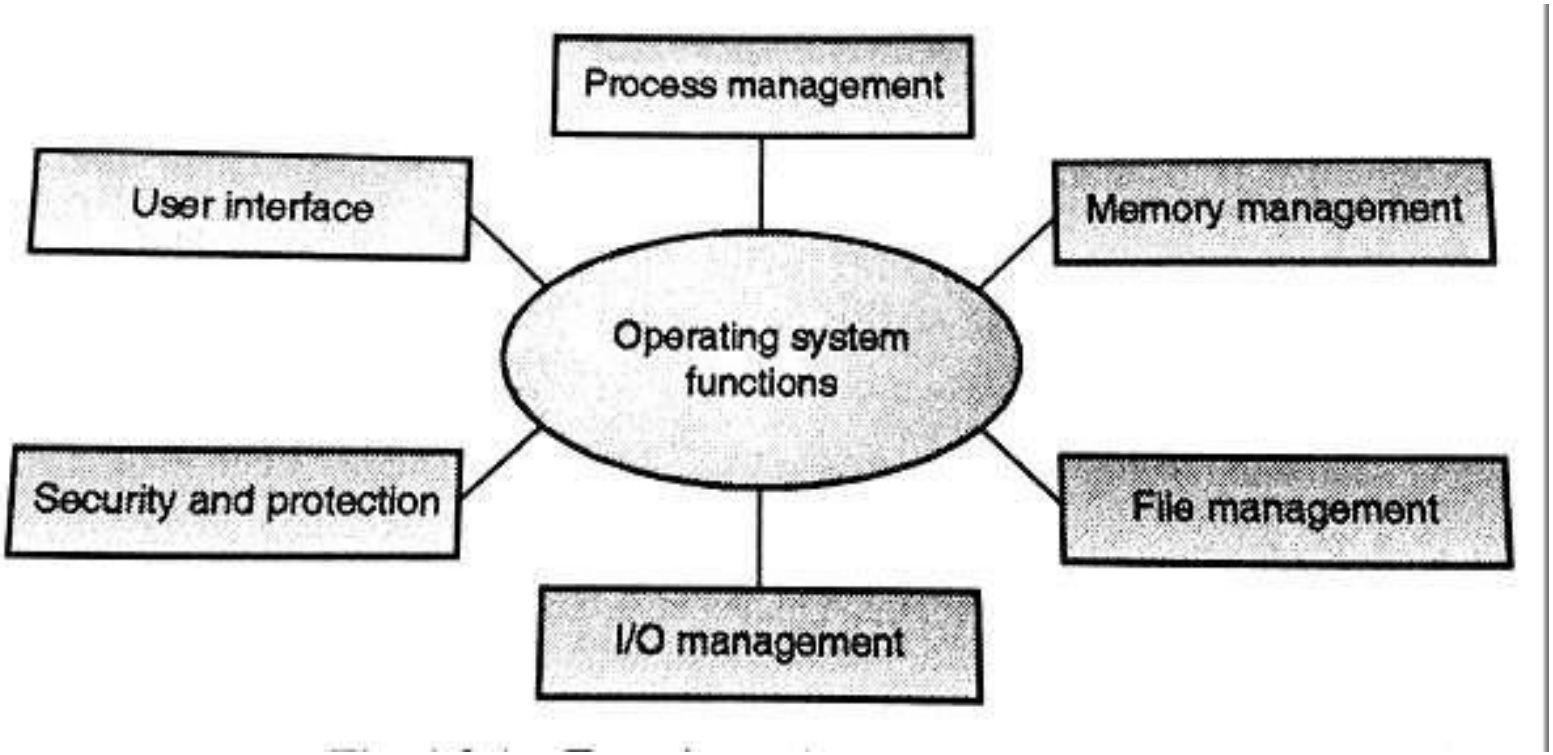
Mobile Operating Systems – (Late 1990s to Early 2000s)

- Invention of smartphones create a big revolution in software industry, To handle the operation of smartphones , a special type of operating systems were developed. Some of them are : iOS and Android etc. These operating systems were optimized with the time and became more powerful.

AI Integration – (2010s to ongoing)

- With the growth of time, Artificial intelligence came into picture.Operating system integrates features of AI technology like Siri, Google Assistant, and Alexa and became more powerful and efficient in many way. These AI features with operating system create a entire new feature like voice commands, predictive text, and personalized recommendations.

Functions of Operating System



Process Management

- In a multi-programming environment, the OS decides the order in which processes have access to the processor, and how much processing time each process has. This function of OS is called **Process Scheduling**.

- An operating system **manages the processors** work by allocating various jobs to it and ensuring that each process receives enough time from the processor to function properly.

Process Management

- The program that keeps a track of the status of processes is known as ‘Traffic Controller”.
- Allocates the CPU that is a processor to a process. De-allocates processor when a process is no more required.

Device Management

- An OS manages device communication via its respective drivers.
- Keeps track of all devices connected to the system. designates a program responsible for every device known as the Input/Output controller.
- Decides which process gets access to a certain device and for how long. Allocates devices effectively and efficiently. De-allocates devices when they are no longer required.
- An OS receives the requests from these devices, performs a specific task, and communicates back to the requesting process.

File Management

- A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files.
- An Operating System carries out the following file management activities.
  - It keeps track of where information is stored, user access settings, the status of every file, and more. These facilities are collectively known as the file system.
  - An OS keeps track of information regarding the creation, deletion, transfer, copy, and storage of files in an organized way.
  - It also maintains the integrity of the data stored in these files, including the file directory structure, by protecting against unauthorized access

User Interface or Command Interpreter

- The user interacts with the computer system through the operating system. Hence OS act as an interface between the user and the computer hardware. This user interface is offered through a set of commands or a **graphical user interface (GUI).**
- Through this interface, the user makes interaction with the applications and the machine hardware

Security

- The operating system uses password protection to protect user data and similar other techniques. it also prevents unauthorized access to programs and user data.
  - Protection against unauthorized access through login.
  - Protection against intrusion by keeping firewall active.
  - Protecting the system memory against malicious access.
  - Displaying messages related to system vulnerabilities

- **Job Accounting**
  - The operating system Keeps track of time and resources used by various tasks and users, this information can be used to track resource usage for a particular user or group of users.

- **Error-Detecting Aids**
  - The operating system constantly monitors the system to detect errors and avoid malfunctioning computer systems. From time to time, the operating system checks the system for any external threat or malicious software activity.

..Cont

- **Network Management**
  - The OS provides network connectivity and manages communication between computers on a network
- **Performs Basic Computer Tasks**
  - The management of various peripheral devices such as the mouse, keyboard, and printer is carried out by the operating system.
- **Coordination Between Other Software and Users**
  - Operating systems also coordinate and assign **interpreters**, **compilers**, **assemblers**, and other software to the various users of the computer systems.

Commonly Used Operating System

- There are various types of Operating Systems used throughout the world and this depends mainly on the type of operations performed. These Operating Systems are manufactured by large multinational companies like Microsoft, Apple, etc. Let's look at the few most commonly used OS in the real world:
- Windows
- UNIX
- LINUX
- BOSS
- SOLARIS